

LAPORAN AKHIR UAS PROJECT PEMROGRAMAN DASAR

“MEALMATCH : APLIKASI REKOMENDASI MAKANAN”



Disusun Oleh :

- | | |
|---------------------------------|--------------------|
| 1. Dalilah Putri Rahma | 25031554239 |
| 2. Nabila Putri Misbahul | 25031554172 |
| 3. Rafli Akbar Maulana | 25031554236 |

**PROGRAM STUDI S1 SAINS DATA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS NEGERI SURABAYA
2025-2026**

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan ini dengan judul “Perancangan dan Implementasi Aplikasi Rekomendasi Makanan (MealMatch) dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas mata kuliah/praktikum serta sebagai bentuk dokumentasi dari proses perancangan dan pembuatan aplikasi rekomendasi makanan. Aplikasi ini bertujuan untuk membantu pengguna dalam memperoleh rekomendasi makanan beserta informasi nilai gizi berdasarkan data yang telah disediakan.

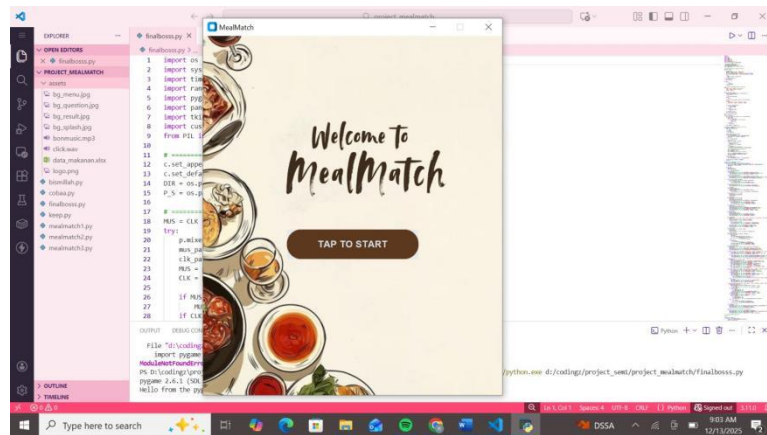
Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat keterbatasan dan kekurangan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi kesempurnaan laporan ini di masa yang akan datang.

Akhir kata, penulis mengucapkan terima kasih kepada semua pihak yang telah membantu dan memberikan dukungan dalam proses penyusunan laporan ini. Semoga laporan ini dapat memberikan manfaat bagi pembaca dan pihak yang membutuhkan.

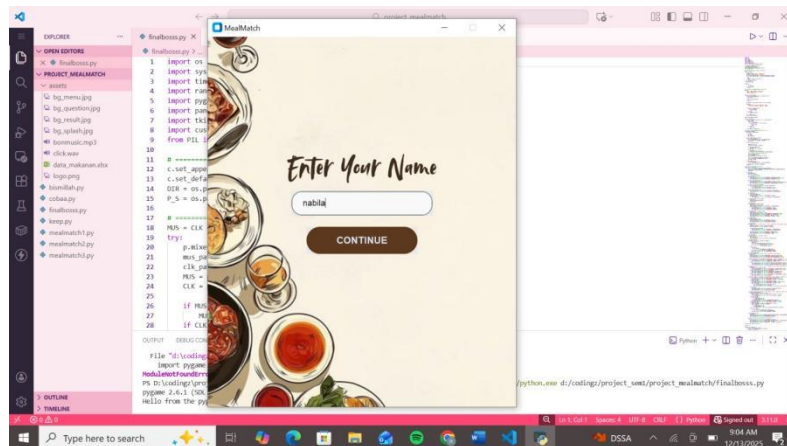
DAFTAR ISI

HALAMAN JUDUL	i
KATA PENGANTAR	ii
DAFTAR ISI	iii
DAFTAR GAMBAR	iv
Gambar 5.1 Tampilan Halaman Awal (Splash Screen) Aplikasi MealMatch .	iv
Gambar 5.2 Tampilan Entry Nama Pengguna	iv
Gambar 5.3 Tampilan Pertanyaan Selera Rasa	iv
Gambar 5.4 Tampilan Pertanyaan Budget	v
Gambar 5.5 Tampilan Pertanyaan Kondisi	v
Gambar 5.6 Tampilan Pertanyaan Porsi	v
Gambar 5.7 Tampilan Rekomendasi Makanan	vi
BAB 1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	1
1.3 Tujuan Pembuatan Aplikasi	2
1.4 Manfaat Pembuatan Aplikasi	2
BAB 2 ANALISIS DAN PERANCANGAN	3
2.1 Analisis Kebutuhan Aplikasi	3
2.2 Flowchart	4
2.3 Sketsa Desain Antar Muka	5
BAB 3 IMPKLEMENTASI	6
3.1 Implimentasi Kode	6
PENUTUP	19

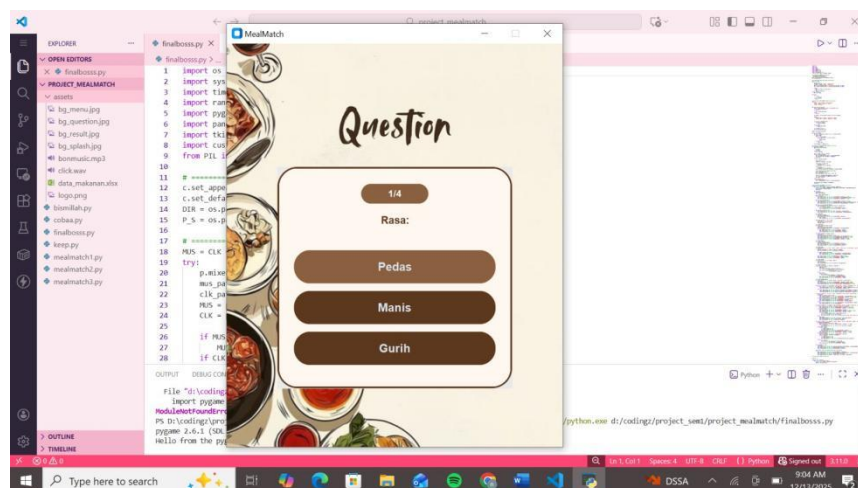
DAFTAR GAMBAR



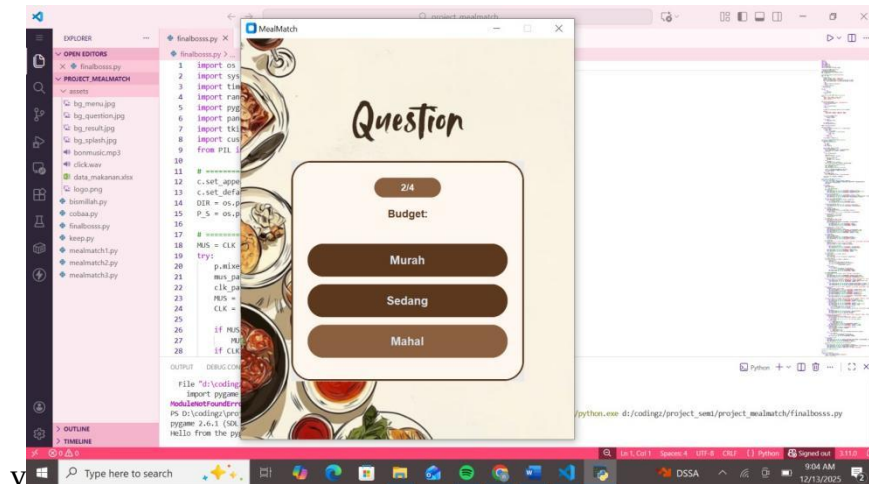
Gambar 5.1 Tampilan Halaman Awal (Splash Screen) Aplikasi MealMatch



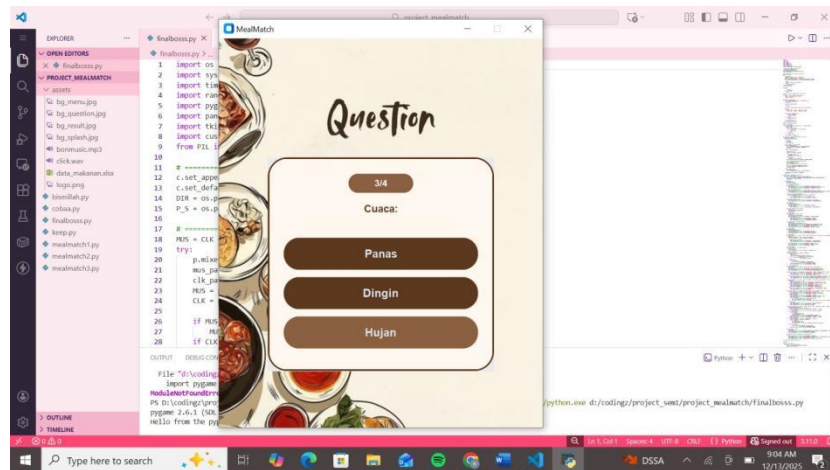
Gambar 5.2 Tampilan Entry Nama Pengguna



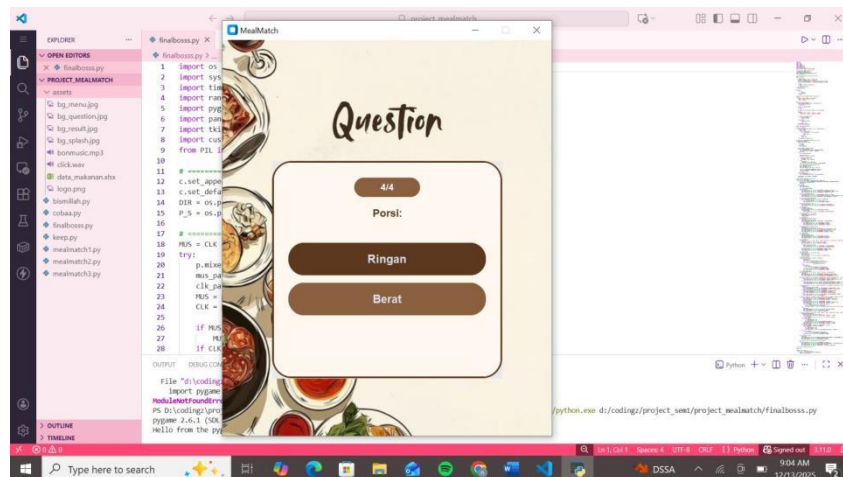
Gambar 5.3 Tampilan Pertanyaan Selera Rasa



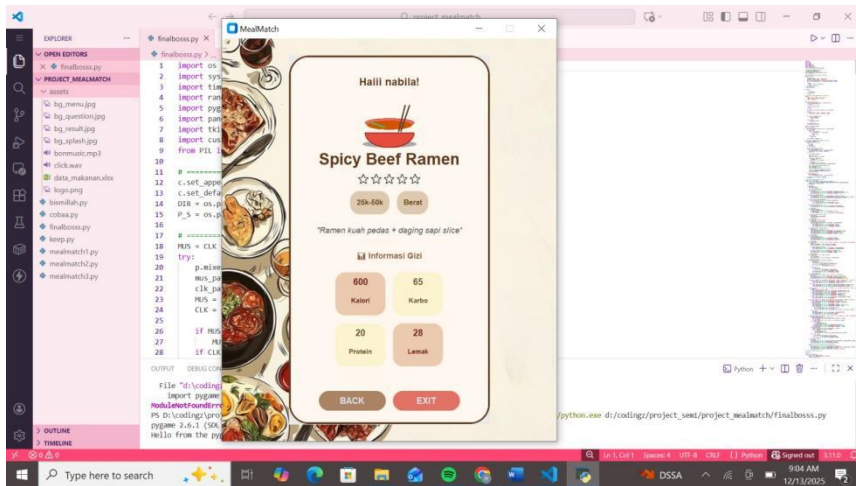
Gambar 5.4 Tampilan Pertanyaan Budget



Gambar 5.5 Pertanyaan Kondisi



Gambar 5.6 Tampilan Pertanyaan Porsi



Gambar 5.7 Tampilan Rekomendasi makanan

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Anak kos sering kali menghadapi kesulitan dalam menentukan pilihan makanan yang sesuai dengan kondisi dan kebutuhan sehari-hari. Faktor seperti keterbatasan anggaran, perbedaan selera makanan, serta pengaruh kondisi cuaca menjadi pertimbangan penting dalam memilih makanan. Namun, tidak semua anak kos memiliki informasi dan referensi yang cukup untuk menentukan pilihan makanan yang tepat secara cepat dan efisien.

Selain itu, kurangnya informasi mengenai kandungan gizi pada makanan yang dikonsumsi dapat berdampak pada pola makan yang kurang seimbang. Padahal, asupan gizi yang baik sangat penting untuk menjaga kesehatan dan menunjang aktivitas sehari-hari, terutama bagi anak kos yang memiliki mobilitas dan aktivitas yang tinggi.

Berdasarkan permasalahan tersebut, dikembangkan sebuah aplikasi rekomendasi makanan bernama MealMatch. Aplikasi ini dirancang untuk membantu anak kos dalam memperoleh rekomendasi makanan berdasarkan kondisi cuaca, selera makanan, serta anggaran yang dimiliki. Aplikasi MealMatch juga menyediakan informasi nilai gizi dari setiap rekomendasi makanan sehingga pengguna dapat mempertimbangkan aspek kesehatan dalam menentukan pilihan makanan.

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, maka rumusan masalah dalam pembuatan aplikasi MealMatch adalah sebagai berikut :

1. Bagaimana merancang dan mengimplementasikan aplikasi rekomendasi makanan untuk membantu anak kos dalam memilih makanan sehari-hari?
2. Bagaimana aplikasi dapat memberikan rekomendasi makanan berdasarkan kondisi cuaca, selera makanan, dan anggaran pengguna?
3. Bagaimana aplikasi dapat menyajikan informasi nilai gizi makanan secara sederhana dan mudah dipahami?

1.3 Tujuan Pembuatan Aplikasi

Tujuan dari pembuatan aplikasi MealMatch adalah sebagai berikut :

1. Merancang dan mengimplementasikan aplikasi rekomendasi makanan berbasis Python.
2. Membantu anak kos dalam menentukan pilihan makanan yang sesuai dengan cuaca, selera, dan budget yang dimiliki.
3. Menyediakan informasi nilai gizi sebagai bahan pertimbangan dalam memilih makanan.

1.4 Manfaat Pembuatan Aplikasi

Adapun manfaat dari pembuatan aplikasi MealMatch adalah sebagai berikut :

1. Memudahkan anak kos dalam memilih makanan sehari-hari secara cepat dan praktis.
2. Membantu pengguna mengelola anggaran makan dengan lebih efektif.
3. Memberikan informasi nilai gizi untuk mendukung pola makan yang lebih sehat

BAB 2

ANALISIS DAN PERANCANGAN

2.1 Analisis Kebutuhan Aplikasi

Analisis kebutuhan aplikasi MealMatch dilakukan untuk mengidentifikasi permasalahan yang dihadapi pengguna serta menentukan solusi yang tepat melalui pengembangan sistem. Analisis ini bertujuan agar aplikasi yang dibuat benar-benar sesuai dengan kebutuhan pengguna dan dapat berfungsi secara optimal. Dalam penelitian ini, analisis kebutuhan ditinjau dari dua aspek utama, yaitu kebutuhan pengguna dan kebutuhan sistem.

2.1.1 Kebutuhan Pengguna

Kebutuhan pengguna berangkat dari permasalahan yang sering dialami mahasiswa dalam menentukan pilihan makanan sehari-hari. Aktivitas perkuliahan yang padat, keterbatasan waktu, serta banyaknya pilihan makanan sering membuat mahasiswa mengalami kebingungan dalam menentukan menu yang sesuai. Permasalahan ini terlihat sederhana, namun dapat berdampak pada efisiensi waktu dan kenyamanan dalam menjalani aktivitas sehari-hari.

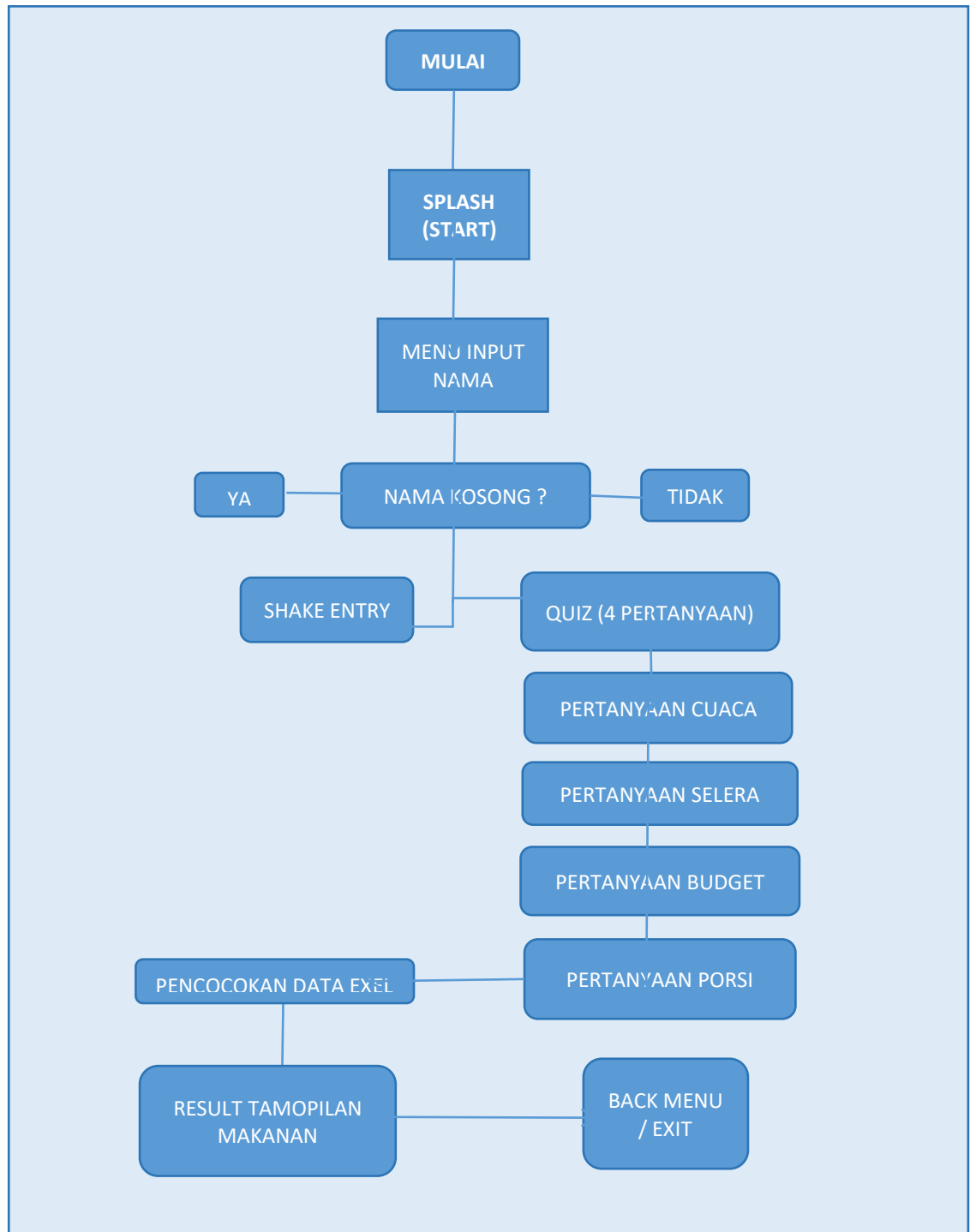
Aplikasi MealMatch dirancang sebagai solusi untuk membantu pengguna dalam mengambil keputusan terkait pilihan makanan. Pengguna membutuhkan sebuah aplikasi yang mudah digunakan, tidak memerlukan proses yang rumit, serta mampu memberikan rekomendasi secara cepat. Dalam memenuhi kebutuhan tersebut, MealMatch mempertimbangkan beberapa parameter yang umum digunakan mahasiswa.

2.1.2 Kebutuhan sistem

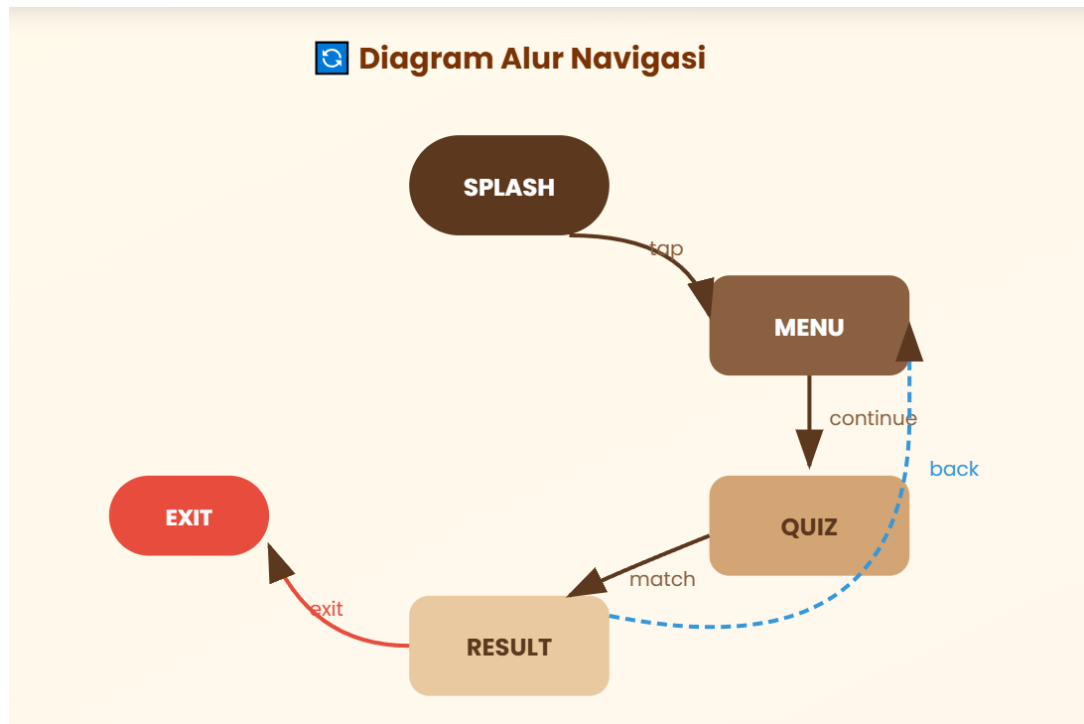
Kebutuhan sistem berkaitan dengan bagaimana aplikasi bekerja dalam mengolah data dan menghasilkan rekomendasi makanan. Sistem harus mampu menerima input dari pengguna, memproses data tersebut, serta menampilkan hasil rekomendasi secara tepat. Untuk

memenuhi kebutuhan tersebut, MealMatch menerapkan metode *rule-based matching* dalam proses penentuan rekomendasi.

2.2 Flowchart



2.3 Sketsa Desain Antarmuka



BAB 3

IMPLEMENTASI CODE

3.1 Implementasi Code

===== IMPORT =====

```
import os
import sys
import time
import random
import openpyxl
import pygame as p
import customtkinter as c
from PIL import Image as PILImage, ImageTk
import os → buat urusan path/folder, cek file ada/tidak.
import sys → buat sys.exit() (keluar aplikasi).
import time → buat delay kecil (shake entry)
import random → buat variasi gambar (tekstur nasi/sprinkle dll).
import openpyxl → baca Excel .xlsx.
import pygame as p → modul audio (mixer & sound).
import customtkinter as c → komponen GUI (frame, button, entry).
from PIL ... → PIL untuk baca/resize image, ImageTk buat icon window.
```

===== CONFIG =====

```
c.set_appearance_mode("light")
c.set_default_color_theme("blue")
DIR = os.path.dirname(os.path.abspath(_file_))
P_S = os.path.join
set_appearance_mode("light") → mode tampilan terang.
set_default_color_theme("blue") → tema default biru (walau kamu
override banyak warna manual).
DIR = ...(_file_) → ambil folder tempat file .py ini berada (biar path asset
konsisten).
P_S = os.path.join → alias biar gampang nulis join path.
```

===== audio init (safe) =====

```
MUS = CLK = None
```

Bikin variabel global audio: MUS (musik), CLK (klik). Default None biar aman.

```
try:
    p.mixer.init()
```

Coba nyalain audio mixer pygame. Kalau device audio error → akan masuk except.

```
mus_path = P_S(DIR, "assets", "bonmusic.mp3")
clk_path = P_S(DIR, "assets", "click.wav")
```

Bikin path file musik & klik di folder assets.

```
MUS = p.mixer.Sound(mus_path) if os.path.exists(mus_path)
else None
CLK = p.mixer.Sound(clk_path) if os.path.exists(clk_path)
else None
```

Kalau file ada → load jadi Sound.

Kalau nggak ada → tetap None (biar program nggak crash).

```
if MUS:
    MUS.set_volume(0.25); MUS.play(-1)
```

Kalau musik berhasil diload:

set volume 25%

play(-1) artinya loop terus.

```
if CLK:
    CLK.set_volume(0.45)
```

Kalau sound klik ada → volume 45%.

```
except Exception :
    MUS = CLK = None
```

Kalau ada masalah apapun → disable audio total (tetap jalan tanpa suara).

`play()` (bunyi klik)

```
def play():
    try:
        if CLK:
            CLK.play()
    except Exception:
        pass
```

Fungsi kecil untuk mainin sound klik tiap tombol.

Aman: kalau error, diabaikan.

===== DATA LOADER =====

```
KEYS = [  
    "nama", "rasa", "budget", "cuaca", "porsi",  
    "kalori", "karbo", "protein", "lemak",  
    "harga", "deskripsi",  
]
```

Ini daftar kolom wajib yang kamu butuhin dari Excel.

```
load_foods_from_excel()
```

```
def load_foods_from_excel():  
    file = os.path.join(DIR, "assets", "data_makanan.xlsx")
```

Tentukan lokasi file excel.

```
if not os.path.exists(file):  
    return []
```

Kalau file nggak ada → return list kosong (FOODS jadi kosong).

```
try:  
    wb = openpyxl.load_workbook(file, data_only=True)  
    sheet = wb.active
```

Load workbook Excel.

data_only=True → ambil hasil nilai final, bukan rumus.

wb.active → sheet aktif.

```
rows = list(sheet.iter_rows(values_only=True))  
if not rows: return []
```

Ambil semua baris jadi list.

Kalau kosong → return [].

```
raw_headers = rows[0]  
headers = [str(h).lower().strip() if h is not None  
else "" for h in raw_headers]
```

Baris pertama dianggap header.

Dibersihkan: jadi lowercase, strip spasi, None jadi string kosong.

```
mapping = {  
    "kalori (kkal)": "kalori", "karbo (g)": "karbo",  
    "protein (g)": "protein", "lemak (g)": "lemak",  
}
```

Kalau excel pakai nama kolom lama, diseragamkan.

`final_headers = [mapping.get(h, h) for h in headers]`

Terapkan mapping: kalau header ada di mapping → diganti, kalau tidak → pakai aslinya.

```
for k in KEYS:
    if k not in final_headers:
        print(f"Kolom hilang: {k}")
        return []
```

Validasi semua kolom wajib ada.

Kalau ada yang kurang → print debug dan return [].

```
data_list = []
for row_values in rows[1:]:
```

Siapin list hasil.

Loop semua baris data (skip header).

```
record = {}
row_dict = {}
```

record sebenarnya nggak kepakai (bisa dihapus, tapi tidak ganggu).

row_dict untuk mapping header → nilai.

```
for i, header in enumerate(final_headers):
    if i < len(row_values):
        val = row_values[i]
        row_dict[header] = val if val is not
None else ""
    else:
        row_dict[header] = ""
```

Untuk setiap kolom header:

ambil nilai di row yang sama index.

kalau None → ganti "" (mirip fillna).

kalau row lebih pendek dari header → isi "".

```
clean_record = {k: row_dict.get(k, "") for k in KEYS}
data_list.append(clean_record)
```

Ambil hanya kolom yang ada di KEYS (kolom lain dibuang).

Masukkan ke list.

return data_list

Kembalikan list of dict.

```
except Exception as e:
    print("Error loading excel:", e)
    return []
```

Kalau error baca excel → return [].

```
FOODS = load_foods_from_excel()
```

Load data makanan sekali di awal program.

```
get_match(answer)
```

```
def get_match(answer):
    ans = {k: str(v).strip().lower() for k, v in
answer.items()}
```

Normalisasi jawaban user: jadikan lowercase dan rapihin spasi.

```
for food in FOODS:
    cocok = True
```

Loop tiap makanan di excel, asumsi cocok dulu.

```
for k, v in ans.items():
    f = str(food.get(k, "")).strip().lower()
    if f != v:
        cocok = False
        break
```

Cek satu per satu field jawaban (rasa/budget/cuaca/porsi).
Kalau beda → cocok=False, stop cek makanan itu.

```
if cocok:
    return food
```

Kalau cocok semua → return makanan itu.

```
return FOODS[0] if FOODS else {}
```

Kalau tidak ada yang cocok:
ambil makanan pertama sebagai default
kalau FOODS kosong → return {}

===== SAFE BACKGROUND HELPER =====

```
bg(parent, filename)
```

```
def bg(parent, filename):  
    app = parent.winfo_toplevel()
```

Ambil window utama (App) dari widget parent.

```
try:  
    W = app.W  
    H = app.H  
except Exception:  
    W, H = 500, 600
```

Ambil ukuran window dari atribut App.W/H.

Kalau belum ada → fallback 500×600.

```
path = P_S(DIR, "assets", filename)  
if not os.path.exists(path):  
    lbl = c.CTkLabel(parent, text="", fg_color="#FFF6EE")  
    lbl.place(x=0, y=0, relwidth=1, relheight=1)  
    return lbl
```

Kalau file background tidak ada → buat label warna polos jadi background.

```
try:  
    img = PILImage.open(path)
```

Buka gambar dengan PIL.

```
target_ratio = W / H  
img_ratio = img.width / img.height
```

Hitung ratio target vs ratio gambar.

```
if img_ratio > target_ratio:  
    new_height = H  
    new_width = int(H * img_ratio)  
else:  
    new_width = W  
    new_height = int(W / img_ratio)
```

Resize dengan mempertahankan aspek rasio (fit-to-cover).

```
img = img.resize((new_width, new_height), PILImage.LANCZOS)
```

Resize kualitas bagus (LANCZOS).

```
left = (new_width - W) // 2
top = (new_height - H) // 2
img = img.crop((left, top, left + W, top + H))
```

Crop tengah supaya pas 500×600 (atau ukuran window).

```
ctk_img = c.CTkImage(light_image=img, size=(W, H))
lbl = c.CTkLabel(parent, image=ctk_img, text="")
lbl.image = ctk_img
```

Convert jadi CTkImage dan tempel ke label.

lbl.image = ... penting supaya image nggak “kehapus” oleh garbage collector.

```
except Exception as e:
    print("WARN load bg:", e)
    lbl = c.CTkLabel(parent, text="", fg_color="#FFF6EE")
```

Kalau gagal load gambar → fallback label polos.

```
lbl.place(x=0, y=0, relwidth=1, relheight=1)
return lbl
```

Taruh background memenuhi frame.

===== FOOD VISUAL =====

FoodArtist (Canvas gambar makanan)

```
class FoodArtist(c.CTkCanvas):
    def __init__(self, master, food_name, width=200,
height=130):
        super().__init__(master, width=width, height=height,
bg="#FFF8F0", highlightthickness=0)
        self.food_name = food_name.lower()
        self.draw()
```

class FoodArtist(...) → bikin canvas khusus untuk gambar makanan.

super().__init__(...) → set ukuran canvas + warna background canvas.

self.food_name = ...lower() → nama makanan dibikin lowercase biar gampang dicek pakai in.

self.draw() → langsung gambar begitu object dibuat.

```
def draw(self):
    self.delete("all")
    name = self.food_name
```

`delete("all")` → bersihin canvas dulu supaya gambar lama hilang.
`name = ...` → variabel bantu biar ringkas saat banyak if.

Fungsi bantu gambar (dipakai ulang)

```
def draw_plate():
    self.create_oval(30, 95, 170, 125, fill="#D0D0D0",
outline="")
    self.create_oval(30, 85, 170, 120, fill="white",
outline="#B0B0B0", width=2)
    self.create_oval(40, 90, 160, 115, fill="#F5F5F5",
outline="")
```

`draw_plate()` → gambar piring (bayangan, rim, inner).

```
def draw_bowl(color="#E74C3C"):
    self.create_oval(40, 110, 160, 125, fill="#A9A9A9",
outline="")
    self.create_arc(45, 60, 155, 125, start=180, extent=180,
fill=color, outline="#922B21", width=2)
    self.create_oval(45, 60, 155, 80, fill="#EC7063",
outline="#922B21")
```

`draw_bowl()` → gambar mangkok + warna mangkok bisa beda.

```
def draw_rice_mound(color="#FFFDD0"):
    self.create_oval(60, 65, 140, 105, fill=color,
outline="#CCC")
    for _ in range(8):
        rx, ry = random.randint(70, 130), random.randint(75,
95)
        self.create_oval(rx, ry, rx+2, ry+2, fill="#DDD",
outline="")
```

`draw_rice_mound()` → gambar nasi + titik-titik tekstur random.

```
def draw_egg_sunny(x=110, y=75):
    self.create_oval(x-20, y-15, x+20, y+15, fill="white",
outline="#EEE")
    self.create_oval(x-10, y-8, x+10, y+10, fill="#FFD700",
outline="#DAA520")
```

`draw_egg_sunny()` → gambar telur ceplok.
Percabangan gambar berdasarkan nama makanan

```

if "rice bowl" in name:
    draw_bowl(color="#F5CBA7")
    self.create_oval(60, 75, 140, 115, fill="#FFFDD0",
outline="")

```

Kalau makanan mengandung “rice bowl” → gambar mangkok + nasi.

```

elif "nasi" in name or "rice" in name:
    draw_plate()
    draw_rice_mound(color="#FFFDD0")
    draw_egg_sunny()

```

Kalau mengandung “nasi/rice” → piring + nasi + telur.

```

elif "ayam" in name or "chicken" in name:
    draw_plate()
    draw_rice_mound(color="#FFFDD0")
    self.create_oval(70, 60, 130, 105, fill="#D35400",
outline="#B4A296")
    self.create_rectangle(125, 85, 155, 95, fill="#F5CBA7",
outline="")

```

Kalau “ayam/chicken” → nasi + ayam (oval) + tulang (rectangle).

```

elif "bubur" in name:
    draw_bowl(color="#3498DB")
    self.create_oval(50, 65, 150, 85, fill="#FDFEFE",
outline="")

```

Kalau “bubur” → mangkok + bubur.

```

elif any(x in name for x in ["mie", "bakso", "ramen",
"wonton kuah", "tom yum odeng", "spicy tteokbokki"]):
    draw_bowl()

```

Kalau masuk kategori kuah/mie → gambar mangkok kuah.

```

else:
    self.create_text(100, 90, text="?", font=("Arial", 40,
"bold"), fill="white")

```

Kalau nggak dikenali → gambar default tanda tanya.

> Intinya: FoodArtist itu “ilustrator otomatis” berdasarkan keyword nama makanan.

===== PAGES =====

Splash

```
class Splash(c.CTkFrame):
    def __init__(self, master, go_cb):
        super().__init__(master, fg_color="transparent")
        bg(self, "bg_splash.jpg")
```

Bikin halaman splash + pasang background.

```
c.CTkButton(..., command=lambda: (play(),
go_cb())).place(...)
```

Tombol “TAP TO START”: bunyi klik lalu pindah ke Menu via callback go_cb().

Menu

```
class Menu(c.CTkFrame):
    def __init__(self, master, quiz_cb):
        super().__init__(master, fg_color="transparent")
        bg(self, "bg_menu.jpg")
```

Halaman menu, pasang background, siap masuk kuis.

```
self.entry_nama = c.CTkEntry(self, placeholder_text="type
here", ...)
```

Input untuk nama user.

```
c.CTkButton(..., command=lambda: (play(),
self.simpan_nama(quiz_cb))).place(...)
```

Tombol CONTINUE: bunyi klik lalu simpan nama → lanjut kuis.

```
def simpan_nama(self, quiz_cb):
    global nama_user
    nama_user = self.entry_nama.get().strip()
    if nama_user == "":
        self.shake_entry(); return
    quiz_cb()
```

Ambil isi entry, rapihin spasi.

Kalau kosong → shake.

Kalau valid → panggil callback masuk Quiz.

```
def shake_entry(self):
    for o in [0.02, -0.02, 0.01, -0.01, 0]:
```

Animasi “goyang” dengan geser posisi entry beberapa kali.

```
class Quiz(c.CTkFrame):
    def __init__(self, master, res_cb):
        super().__init__(master, fg_color="transparent")
        bg(self, "bg_question.jpg")
```

Halaman kuis dengan background.

```
self.Q = [
    ("rasa", "Rasa:", [ ("pedas", "Pedas"), ... ]),
    ("budget", "Budget:", [ ("murah", "Murah"), ... ]),
    ("cuaca", "Cuaca:", [ ("panas", "Panas"), ... ]),
    ("porsi", "Porsi:", [ ("ringan", "Ringan"), ("berat", "Berat") ])
]
```

Bank pertanyaan: setiap item = (key, judul, pilihan).

```
self.idx = 0
self.ans = {}
self.res_cb = res_cb
```

idx buat posisi pertanyaan ke berapa.

ans nyimpen jawaban user.

res_cb callback untuk pindah Result setelah selesai.

```
self.load_question()
```

Pertama kali langsung load pertanyaan pertama.

```
def load_question(self):
    ... destroy tombol lama ...
    k, title, opts = self.Q[self.idx]
```

Bersihkan tombol lama, ambil data pertanyaan sesuai index.

```
btn = c.CTkButton(..., command=lambda v=val, key=this_key:
self.next_answer(k, v))
```

Buat tombol pilihan; kalau diklik → simpan jawaban.

```
def next_answer(self, key, val):
    self.ans[key] = val
    self.idx += 1
    if self.idx >= len(self.Q):
        self.res_cb(self.ans)
    else:
        self.load_question()
```

Simpan jawaban.

Kalau semua pertanyaan selesai → kirim jawaban ke Result.

Kalau belum → lanjut pertanyaan berikutnya.

```
class Result(c.CTkFrame):
    def __init__(self, master, ans, menu_cb):
        super().__init__(master, fg_color="transparent")
        bg(self, "bg_result.jpg")
```

Halaman hasil + background.

```
M = get_match(ans) or {}
```

Cari makanan yang cocok dari jawaban quiz.

```
food_name = M.get("nama", "Makanan").title()
self.art_canvas = FoodArtist(self.content_frame,
                              food_name, ...)
```

Ambil nama makanan.

Panggil FoodArtist buat gambar makanannya.

```
self.create_badge(info_box, f"{M.get('harga','-')}", ...)
self.create_badge(info_box, f"{M.get('porsi','-')}", ...)
```

Tampilkan badge info harga & porsi.

```
nuts_data = [("Kalori", ...), ("Karbo", ...),
              ("Protein", ...), ("Lemak", ...)]
```

Data kartu info gizi.

```
c.CTkButton(... command=lambda: (play(), menu_cb()))
c.CTkButton(... command=lambda: sys.exit())
```

BACK: balik ke menu

EXIT: keluar app

```
self.animate_entry()
```

Animasi card hasil naik dari bawah.

===== APP CONTROLLER =====

App

```
class App(c.CTk):
    def __init__(self):
        super().__init__()
        W, H = 500, 600
        self.W, self.H = W, H
        self.geometry(f"{W}x{H}")
```

Bikin window utama ukuran 500x600 dan simpan ke self.W/H.

```
self.resizable(False, False)
self.title("MealMatch")
```

Window tidak bisa di-resize, judul aplikasi diset.

```
self.after(10, self.show_splash)
```

Setelah window siap, tampilkan Splash.

```
def show(self, frame_class, *args):
    if self.current: self.current.destroy()
    self.current = frame_class(self, *args)
    self.current.pack(fill="both", expand=True)
```

Sistem “pindah halaman”: halaman lama dihancurin, halaman baru dibuat.

```
def show_splash(self):
    self.show(Splash, lambda: self.show(Menu, lambda:
self.show(Quiz, self.show_result)))
```

Alur Splash → Menu → Quiz (via callback).

```
def show_result(self, ans):
    self.show(Result, ans, lambda: self.show(Menu, lambda:
self.show(Quiz, self.show_result)))
```

Setelah quiz selesai → tampil Result.

BACK di Result → balik Menu lagi.

Run app

```
App().mainloop()
```

Start loop GUI (aplikasi mulai jalan & nunggu input user).

PENUTUP

Berdasarkan perancangan dan implementasi yang telah dilakukan, aplikasi MealMatch berhasil dibuat sebagai aplikasi rekomendasi makanan berbasis kuis. Aplikasi ini mampu menerima input pengguna, mengolah data makanan, serta menampilkan hasil rekomendasi beserta informasi pendukung secara sederhana dan interaktif.

Dengan adanya aplikasi ini, diharapkan pengguna dapat lebih mudah menentukan pilihan makanan sesuai dengan preferensi dan kondisi yang diinginkan. Meskipun aplikasi masih memiliki keterbatasan, seperti pencocokan data yang sederhana dan jumlah data makanan yang terbatas, aplikasi ini sudah memenuhi tujuan utama yang telah ditetapkan.

Ke depannya, aplikasi MealMatch dapat dikembangkan lebih lanjut dengan menambahkan fitur yang lebih kompleks, seperti sistem rekomendasi berbobot, kategori diet, serta penambahan data makanan agar hasil rekomendasi menjadi lebih bervariasi dan akurat.

Link GitHub untuk akses full code :

<https://github.com/nabilamisbahul/TUGAS-PROJECT-UAS-PEMDAS>